

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Ajay Kumar Reddy	Reg. No.:	192425274
Section / Batch:	Slot :A / 2024	Date:	

Code of Conduct

*I **Ajay Kumar Reddy / 192425274** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Ajay Kumar Reddy** _____

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

Answer:

Introduction

Sentence structure representation is an important part of Natural Language Processing (NLP), especially for grammar checking, syntactic analysis, information extraction, and machine translation. Two important approaches for representing sentence structure are **Context-Free Grammar (CFG) parse trees** and **dependency parsing structures**. Although both describe relationships within a sentence, they represent those relationships in fundamentally different ways.

1. Context-Free Grammar Trees

A Context-Free Grammar represents a sentence using grammatical rules such as:

- $S \rightarrow NP VP$
- $NP \rightarrow Det N$
- $VP \rightarrow V NP$
- $NP \rightarrow Det Adj N$

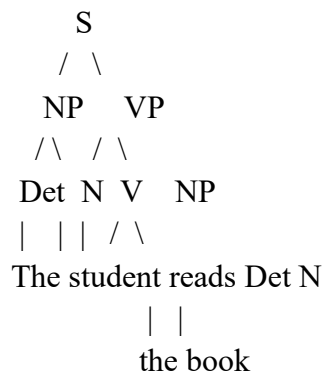
The resulting structure is a **hierarchical parse tree**. The sentence is divided into constituents such as noun phrases (NP), verb phrases (VP), prepositional phrases (PP), and so on.

For example:

Sentence:

"The student reads the book."

A CFG analysis can represent it as:



The tree shows how words combine to form larger grammatical units.

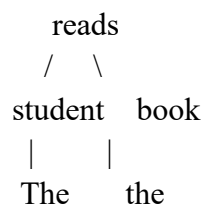
2. Dependency Parsing

Dependency parsing represents a sentence using **direct relationships between words**. Instead of grouping words into phrases, it identifies which word depends on which other word.

For the same sentence:

The student reads the book.

The dependency relationships can be represented as:



Here, **reads** is the main/root verb. "student" is the subject of "reads", and "book" is its object.

Comparison

Aspect	CFG Trees	Dependency Parsing
Basic representation	Constituency tree	Dependency tree/graph
Main focus	Phrase structure	Word-to-word relationships
Structure	Hierarchical constituents	Head-dependent relationships
Grammar categories	NP, VP, PP, etc.	Subject, object, modifier, etc.
Long-distance relationships	Can become difficult to interpret	Usually represented directly
Useful for	Formal grammar and syntactic structure	Semantic relations and NLP applications
Representation complexity	Can become large for long sentences	Generally compact
Head of sentence	Represented indirectly	Explicitly represented

Analysis

CFG trees are very useful when the main requirement is to understand **how words form grammatical phrases**. They are particularly useful in formal grammar, compiler design, traditional syntactic analysis, and grammar-based language processing.

However, for many modern NLP applications, knowing the direct relationship between words is more useful. Dependency parsing explicitly identifies relations such as **subject, object, modifier, and determiner**. This makes it easier to determine who performed an action, what object was affected, and which word modifies another word.

For example:

"The researcher analyzed the medical report carefully."

Dependency parsing can directly identify:

- researcher → subject of analyzed
- report → object of analyzed
- medical → modifier of report
- carefully → modifier of analyzed

This makes dependency structures useful for information extraction, question answering, relation extraction, and semantic analysis.

Justification and Conclusion

Dependency parsing is generally more effective for capturing relationships between words, especially in practical NLP systems. CFG trees provide richer information about phrase hierarchy, but dependency parsing gives direct and compact representations of grammatical relationships.

Therefore, for **grammar-oriented constituency analysis**, CFG is highly suitable. For **modern NLP applications requiring direct word relationships**, dependency parsing is usually more effective.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.

Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

Answer:

Introduction

NLP systems often need to parse user input quickly. In real-time applications, the input may be incomplete, contain errors, or be structurally ambiguous. Traditional top-down parsing and Earley parsing provide different approaches for solving this problem.

1. Top-Down Parsing

Top-down parsing begins with the start symbol of the grammar and attempts to derive the input sentence. For example, if:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

The parser starts with:

S

↓

NP VP

↓

Det N VP

It continues expanding grammar rules until the generated structure matches the input.

Advantages of Top-Down Parsing

1. It is conceptually simple.
2. It follows the grammar from the start symbol.
3. It can be efficient for simple and well-designed grammars.
4. Recursive-descent parsing can be easy to implement.
5. It is suitable when the grammar is small and predictable.

Limitations of Top-Down Parsing

Top-down parsing can experience problems when:

- The grammar contains ambiguity.
- The input is incomplete.
- There are many possible grammar rules.
- Backtracking is required.
- Left-recursive grammar rules are present.

For example, if a user types:

"Show me the transactions..."

and has not completed the sentence, a conventional top-down parser may not have enough information to determine the complete structure.

2. Earley Parsing

Earley parsing is a general parsing algorithm designed to handle a wide range of context-free grammars. It maintains a set of possible parsing states while processing the sentence.

It uses three main operations:

1. **Prediction** – predicts possible grammar rules.
2. **Scanning** – matches grammar symbols with input tokens.
3. **Completion** – completes rules whose constituents have been successfully recognized.

An Earley state is commonly represented using a dot:

$S \rightarrow NP \bullet VP$

The dot indicates how much of the rule has already been recognized.

Comparison

Aspect	Top-Down Parsing	Earley Parsing
Direction	Starts from grammar start symbol	Processes input while maintaining possible states
Ambiguity	Can require backtracking	Can maintain multiple possibilities
Incomplete input	Less suitable	More flexible
Grammar restrictions	Often requires suitable grammar	Handles general CFGs
Left recursion	Can cause problems	Supported
Implementation	Relatively simple	More complex
Dynamic input	Limited	Better suited
Memory usage	Usually lower for simple cases	Can require more memory

Analysis

Top-down parsing works efficiently when the grammar is simple and the input is complete and predictable. However, real-time applications frequently receive incomplete or ambiguous input.

Earley parsing is more flexible because it can maintain several possible interpretations simultaneously. It does not need to commit immediately to a single parse path.

For example:

User

input:

"Book a flight from Chennai..."

The sentence is incomplete because the destination has not yet been supplied. An Earley parser can maintain the possible grammatical states while waiting for additional input.

Similarly, for an ambiguous input, Earley parsing can retain multiple possible parses rather than immediately rejecting the sentence.

Justification and Conclusion

For a real-time application involving incomplete, ambiguous, or dynamically changing input, Earley parsing is more suitable than traditional top-down parsing.

Top-down parsing remains useful for simple and predictable grammars because it is easier and can be faster in controlled situations. However, Earley's generality and ability to manage multiple possible parsing states make it better for dynamic NLP environments.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

Answer:

Introduction

Natural language is inherently ambiguous. A single sentence can have more than one valid syntactic interpretation. Consider:

"She saw the man with a telescope."

This sentence has at least two interpretations:

1. She used a telescope to see the man.
2. She saw a man who had a telescope.

The main challenge is deciding which interpretation is most likely in context.

1. CFG-Based Parsing

A Context-Free Grammar can represent multiple valid parse trees for an ambiguous sentence.

For example:

Interpretation 1:

She -- saw -- [the man] -- [with a telescope]

Interpretation 2:

She -- saw -- [the man with a telescope]

CFG can recognize both structures if the grammar permits them.

However, basic CFG does not inherently provide probabilities. Therefore, it may identify multiple valid structures without knowing which one is more likely.

Advantages of CFG

- Simple formal representation.
- Easy to understand.
- Useful for grammar construction.
- Provides explicit syntactic structures.
- Good for controlled languages.

Limitations of CFG

- Cannot naturally rank competing interpretations.
- Requires manually designed grammar rules.
- Difficult to capture contextual meaning.
- Can become complex for natural language.

2. Probabilistic Context-Free Grammar (PCFG)

PCFG extends CFG by assigning probabilities to grammar rules.

For example:

$VP \rightarrow V NP PP \quad 0.35$

$NP \rightarrow NP PP \quad 0.20$

The parser calculates the probability of different parse trees and selects the most probable interpretation.

For the example:

"She saw the man with a telescope."

PCFG can compare the probabilities of:

She used the telescope

and

The man had the telescope
The interpretation with the higher probability can be selected.

Advantages of PCFG

- 1. Handles structural ambiguity better than basic CFG.
- 2. Can rank multiple parse trees.
- 3. Uses statistical information from training data.
- 4. Provides a more realistic model of language usage.

Limitations of PCFG

- Depends on the quality of training data.
- Rule probabilities may not capture deeper semantic context.
- Basic PCFG models have limited contextual understanding.
- Long-distance relationships can still be difficult.

3. Neural Parsing

Neural parsers use machine-learning models such as Transformer-based architectures to learn patterns from large amounts of language data.
Instead of relying only on manually defined grammar rules, neural parsers learn contextual relationships between words.

For example, the model can consider:

- surrounding words,
- sentence context,
- syntactic patterns,
- semantic information,
- information learned from large datasets.

If the previous sentence was:

"She was carrying a telescope."

then:

"She saw the man with a telescope."

is more likely to mean that **she used the telescope**.

This contextual ability gives neural parsers a major advantage over basic CFG and PCFG systems.

Comparison

Aspect	CFG	PCFG	Neural Parsing
Grammar rules	Explicit	Explicit probabilities +	Mostly learned
Handles ambiguity	Limited	Better	Very strong
Context awareness	Low	Moderate	High
Requires training data	Usually no	Yes	Usually large datasets
Semantic understanding	Limited	Limited	Better
Interpretability	High	Moderate	Lower
Scalability	Limited for complex language	Better	High with resources
Real-world performance	Moderate	Good	Generally strongest

Analysis

CFG is excellent for describing possible grammatical structures, but it cannot effectively choose between ambiguous interpretations without additional mechanisms.

PCFG improves this by assigning probabilities to alternative parse trees. However, probabilities based mainly on grammar rules do not always capture the full context.

Neural parsers can use broader contextual information and learned representations. This allows them to make more accurate decisions in complex real-world sentences.

Justification and Conclusion

For **real-world NLP applications**, **neural parsing is generally the most effective approach for ambiguity resolution**, especially when sufficient training data and computational resources are available. However, CFG and PCFG remain valuable because they provide explicit grammatical structures and can be easier to interpret.

Therefore:

- **CFG** → best for explicit grammatical representation.
- **PCFG** → better for probabilistic ambiguity resolution.
- **Neural parsing** → generally best for complex real-world language understanding.

A practical NLP system may also combine neural models with symbolic grammar constraints to achieve both accuracy and grammatical control.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.

Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

Answer:

Introduction

A grammar system must ensure that sentences are grammatically correct. Two important linguistic mechanisms used for this purpose are **feature structures** and **subcategorization frames**. They solve related but different problems.

1. Feature Structures

Feature structures represent grammatical properties associated with words or phrases.

Common features include:

- Number: singular/plural
- Person: first/second/third
- Gender
- Tense
- Case
- Agreement
- Definiteness

For example:

Noun:

Number = singular

Person = third

Verb:

Number = singular

Person = third

Consider:

"The boy runs."

The subject "boy" is singular and third person. Therefore, the verb "runs" must also be singular and third person.

For:

"The boys run."

the noun is plural, so the verb must agree with the plural subject.

Feature structures can enforce this relationship.

Example

Subject:

Number = singular

Person = third

Verb:

Number = singular

Person = third

If the values are compatible, the sentence passes the agreement constraint.

2. Subcategorization Frames

A subcategorization frame specifies the types and number of arguments that a verb requires.

For example:

eat

eat → Subject + Verb + Object

Example:

"The student ate the sandwich."

The verb "ate" accepts an object.

Another verb:

sleep

sleep → Subject + Verb

Example:

"The student slept."

It normally does not require a direct object.

Another example:

give

give → Subject + Verb + Indirect Object + Direct Object

Example:

"The teacher gave the student a book."

The verb requires the appropriate argument structure.

Comparison

Aspect	Feature Structures	Subcategorization Frames
Main purpose	Represent grammatical properties	Represent verb argument requirements
Handles agreement	Excellent	Limited
Handles number/person	Yes	Usually not primary purpose
Handles verb complements	Limited	Excellent
Represents grammatical features	Detailed	Less detailed
Example	Singular subject + singular verb	Verb requires object
Main use	Agreement and unification	Argument structure

Analysis

Feature structures are particularly useful for **subject–verb agreement**.

For example:

Incorrect:

"The students runs."

The subject has:

Number = plural

while the verb has:

Number = singular

The feature values do not match, so the grammar system can reject the sentence.

However, feature structures alone do not necessarily specify what arguments a verb requires.

For example:

"The student gave."

The sentence may be incomplete because "give" generally expects additional arguments in its common transitive/double-object usage.

A subcategorization frame can identify this requirement:

give → Subject + Verb + Indirect Object + Direct Object

Thus, subcategorization frames are highly effective for detecting missing or inappropriate complements.

Justification and Conclusion

If the main requirement is **subject–verb agreement**, feature structures provide better support because they explicitly represent properties such as number and person.

If the requirement is **verb argument structure**, subcategorization frames are more appropriate because they specify what complements a verb expects.

Therefore, neither mechanism completely replaces the other. **Feature structures are better for enforcing grammatical agreement, while subcategorization frames are better for enforcing verb argument structures.**

For a complete grammar system, combining both approaches provides stronger grammatical checking.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Answer:

Introduction

Dependency parsing is widely used in NLP systems to determine relationships between words. For large-scale NLP applications, the parser must be both accurate and computationally efficient. Two major approaches are transition-based parsing and graph-based parsing.

1. Transition-Based Parsing

Transition-based parsing constructs a dependency tree through a sequence of decisions called transitions.

A parser generally maintains structures such as:

- Stack
- Input buffer
- Dependency relations

Common transitions include:

SHIFT

LEFT-ARC

RIGHT-ARC

REDUCE

For example, when processing:

"The student reads books."

the parser progressively moves words between the stack and buffer and creates dependency relationships.

Main characteristic

Transition-based parsing makes local decisions step by step.

A simplified process is:

Input → SHIFT → ARC → REDUCE → SHIFT → ARC → Dependency Tree

Advantages

1. Very fast parsing.
2. Often has approximately linear-time behavior.
3. Requires relatively low computational resources.
4. Suitable for real-time systems.
5. Modern neural transition-based parsers can achieve high accuracy.

Limitations

The major limitation is local decision-making.

An early incorrect decision can affect later decisions. Therefore, errors can propagate through the parsing process.

2. Graph-Based Parsing

Graph-based parsing treats dependency parsing as a global optimization problem.

The possible dependency relationships between words are represented as edges in a graph.

For example:

reads

/ \

student books

The parser evaluates possible dependency structures and selects the tree with the highest overall score.

Instead of making one local decision at a time, it attempts to find the best complete dependency structure.

Advantages

1. Considers global sentence structure.
2. Reduces the effect of individual local decisions.
3. Often provides strong parsing accuracy.
4. Better at considering interactions among multiple dependencies.

Limitations

1. Computationally more expensive.
2. Requires more memory in many implementations.

3. Can be slower than transition-based parsing.
4. Global optimization becomes challenging for large datasets and long sentences.

Comparison

Aspect	Transition-Based Parsing	Graph-Based Parsing
Decision type	Local, sequential	Global
Parsing process	Series of transitions	Optimization over possible trees
Speed	Usually very fast	Generally slower
Memory	Lower	Higher
Error propagation	More likely	Less dependent on early decisions
Global context	Limited traditionally	Strong
Large-scale processing	Highly suitable	More computationally demanding
Real-time applications	Excellent	Less suitable
Accuracy	High with modern models	Often strong, especially globally

Analysis

Transition-based parsing is particularly attractive for large-scale applications because of its speed. It makes one decision at a time and can construct a dependency tree efficiently.

For example, if millions of sentences must be processed, the ability to parse each sentence quickly can significantly reduce total processing time.

However, transition-based systems can suffer from error propagation. If the parser incorrectly attaches a word early in the process, later decisions may be affected.

Graph-based parsing considers the sentence more globally. Instead of committing immediately to individual decisions, it evaluates candidate dependency structures and selects the best-scoring tree. This can improve robustness and accuracy, particularly when complex relationships exist.

However, global optimization can require considerably more computation.

Modern NLP Consideration

Modern neural parsing has reduced the gap between these approaches. Neural transition-based parsers can use contextual representations to make much better local decisions, while neural graph-based parsers can score relationships using powerful learned representations.

Therefore, the choice depends on the application's requirements:

- High speed and large-scale processing → Transition-based parsing
- Global structural accuracy → Graph-based parsing
- Real-time processing → Transition-based parsing
- Complex dependency analysis where computational cost is acceptable → Graph-based parsing

Justification and Conclusion

For a large-scale NLP system requiring fast dependency parsing, transition-based parsing is generally the more suitable choice because it provides efficient and scalable processing with relatively low computational requirements.

Graph-based parsing may provide stronger global reasoning but can require more computational resources.

Therefore, when the primary requirement is speed, scalability, and real-time or high-volume processing, transition-based parsing is preferred. When the priority is global structural accuracy and computational resources are available, graph-based parsing can be advantageous.

Overall Conclusion

The comparative analysis demonstrates that no single NLP technique is best for every situation. The appropriate method depends on the requirements of the application.

- CFG trees provide detailed hierarchical phrase structures, while dependency parsing directly represents relationships between words.
- Top-down parsing is simple and effective for predictable grammars, while Earley parsing is more flexible for ambiguous and incomplete input.
- CFG represents possible structures, PCFG ranks structures probabilistically, and neural parsing generally provides stronger ambiguity handling in real-world applications.
- Feature structures are effective for grammatical agreement, while subcategorization frames are particularly useful for verb argument structures.
- Transition-based parsing emphasizes speed and scalability, while graph-based parsing emphasizes global structural decisions.

Thus, modern NLP systems often benefit from combining symbolic linguistic knowledge, probabilistic methods, and neural models rather than depending on a single parsing technique.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____